Assignment 2

Alvin Svensson & Jakob Persson
as228id@student.lnu.se & *jp223uy@student.lnu.se*

Task 1 **2**
Task 2 **4**
Task 3 **5**
AI Disclosure **7**
References **8**

Task 1

We will run it with independent replications, estimate the measures with 95% confidence intervals, and validate it with analytic bounds and confidence checks.

We will build a discrete event simulation in python following the next-event time advance approach, with the models described by the event graph below. We will run it with independent replications and estimate the measures with 95% confidence intervals. We will validate it with analytic bounds and confidence checks, comparing the simulated results with values that can be computed exactly or bound analytically [1].

Assumptions about the system:

- FIFO queue for the servers
- If S2 is full it is redirected to off site depot just like S1 and S3
- In the rare event of two servers being done at the same time, we prioritize the Server further down in the chain, for example S3 over S2 to help minimize queue.
- The system starts empty and idle

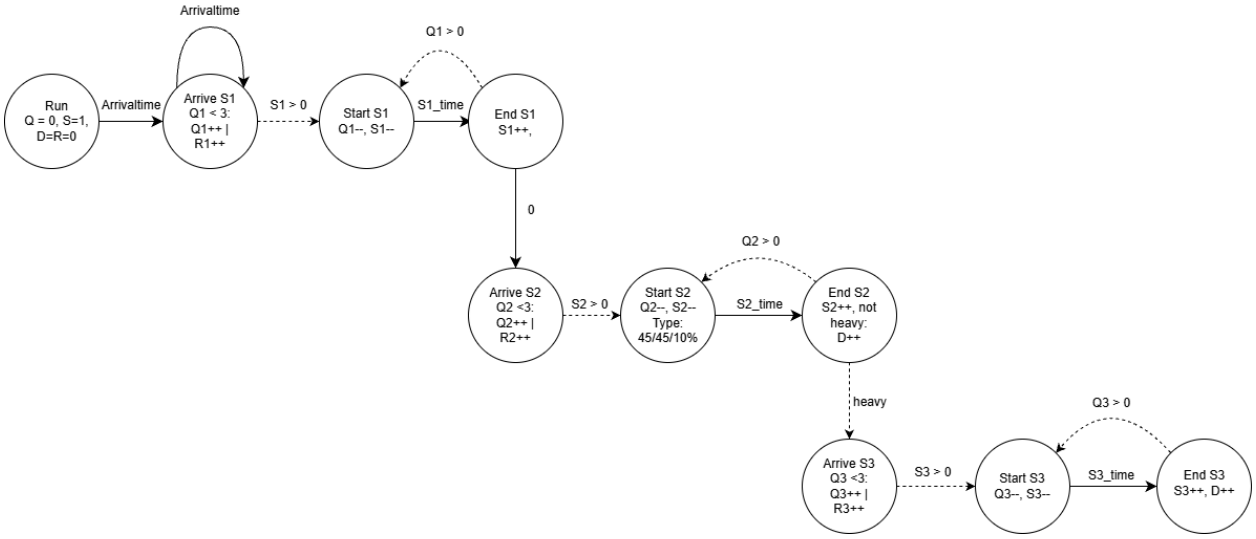


Figure 1 shows the event graph of the maintenance hub.

The event graph has 10 events: Run, Arrive, Start and End event for each of the three stages. The state variables are Qi (i is the stage, Qi ranges from 0-3), Si (i is the stage, Si ranges from 0-1), the redirect counters R1, R2, R3, the class of the AMR in S2, and the departure counter D.

Run initializes the state and schedules the first Arrive S1, which reschedules itself after 80s interarrival time. Each Arrive event adds the AMR to the buffer if there is room, otherwise it redirects the AMR and increments the stage's redirect counter. Start Si runs when Qi > 0 and

Si >0, and schedules End si after the service time. End S1 immediately schedules Arrive S2. End S2 either lets the AMR depart (90%) or schedules Arrive S3 (10%, heavy repair). End S3 increments D.
Each arrow means that when the event at its tail is executed, it schedules the event at its head. Solid arrows are always scheduled and dotted arrows are scheduled only if the condition for the arrow holds [2].

The model uses the next-event time-advance approach: the clock jumps directly to the most imminent event, and the state only changes at those moments. The system state is the collection of variables needed to describe the hub at a given time [1].

State variable	Description
num_in_si	Number of AMRs at a station i including service + buffer
q_si	The buffer queue that holds the AMR's when they are waiting for service. It is a fifo queue with a max size of 3
amr_in_si	Variable that hold the record of the currently served AMR in the system (buffer arrival, hub arrival and total delay)
s2_is_heavy	Boolean that keeps track of if the AMR in stage 2 needs heavy drive repair and therefore, if it should be sent to stage 3
time_last_event	Time of the most recent event, used to compute how long the state has been unchanged when the time-average statistics are updated [1].

Table 1 lists the different state variables. Common variables for different stages are referred to as "si" where i represents the stage number.

In each iteration of the main loop, the timing routine finds the earliest event and advances the clock, with ties going to the station further down the chain [1]. Then the time-weighted areas are updated for the interval that just ended, and the matching event routine changes the state and schedules new events. Arrivals start service, join the buffer or are redirected if the station is full. Departures start the next AMR from the buffer and pass the finished AMR on: S1 to S2, S2 to S3 if heavy and otherwise to the hub and from S3 to the hub. The start events of the event graph are included in these routines, which is equivalent. The run stops at the end of the simulated time after a final update.
The statistics are of two kinds. Counters give the redirect rate (R1 + R2 + R3 divided by the length of data-collection phase) and the throughput (D divided by the same time). Time-weighted

areas give utilization and average queue length. The waiting time per stage is the total buffer delay divided by the number of AMRs that started service there, so immediate starts count as zero. The overall wait and the time in the hub are averaged over the AMRs that completed service in the hub; redirected AMRs are excluded, which should be stated as an assumption.

Task 2

We run a steady-state (non-terminating) simulation. A simulation is terminating when the system has a natural starting condition and a natural end event, such as retail markets that open and close at a specified time. The maintenance hub in this case has neither. A non-terminating system runs concurrently, or at least over a very long period, and a steady-state simulation studies its long run properties, which are not influenced by the initial condition at time 0 [3].

In our case the hub receives AMRs at the constant rate of 0.75 per minute. The measures we collect (redirect rate, utilization, throughput and average waiting time) are long-run states and averages of a continuously operating hub, no results for one particular interval. In a steady-state simulation the run length TE is not given by the problem but is a design choice of the researcher [3], which we show further down.

Our model starts empty and idle, which is not a typical state of the hub, so the first part of each run is biased towards low queues and utilization. We remove this initial warm-up from the statistics and use only observations collected after it, as described in section 2.2.

We use the method of independent replications with deletion of the warm-up period. The reason is that observations inside one run are autocorrelated. For queuing systems the outpost is typically positively autocorrelated, so the usual estimate $\frac{S^2}{n}$ is biased low and the confidence interval would be too short. Observations from different replications are independent, because each replication uses a different random-number stream. Since the hub starts empty and idle, the first part of every run is not part of steady state [3].

To choose the length of the warm-up period T_0 [1], we ran a pilot experiment whose only purpose was to determine T_0. The pilot consisted of R = 20 independent replications of 600 minutes each, with no warm-up and different random seeds.

We then calculated the ensemble averages across the replications in 5-minute time buckets and plotted them as a mean plot [3]. From the plot, we determined where the initial phase had ended by observing where the curves started to only vary around a constant level.

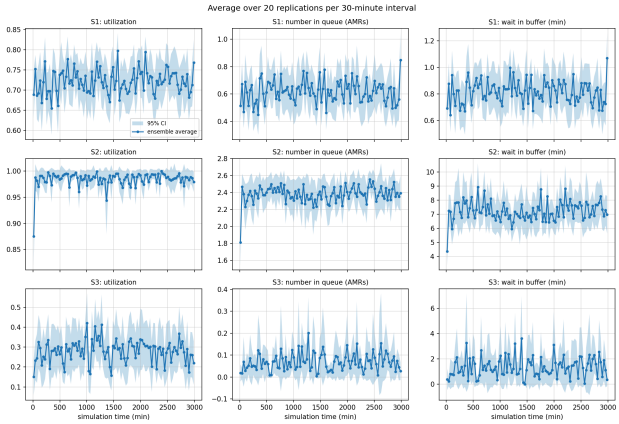


Figure 2 shows the plot of the ensemble of simulations generated by amrPilotSimulation.py

From Figure 2 we picked $T_0 = 120$ minutes, since all the curves were stable around a constant level after the first 30-minute interval; only S2 showed lower values during the first interval (0–30 min).
From the figure 2 we picked t0 = 120 since all the curves where

Task 3

The formulas used to calculate mean, standard deviation and Half-width H [5].

$$\overline{Y} = \frac{1}{n}\sum_{i=1}^{n} Y_i,\ S^2 = \frac{1}{n-1}\sum_{i=1}^{n}\left(Y_i - \overline{Y}\right)^2,\ H_i = t_{a/2,n_i-1}\frac{S_i}{\sqrt{n_i}}$$

Measures	Average	Sample variance	Half-width H	95% CI
Redirection rate (amr/h)	23.93	$3.672\cdot10^{-2}$	0.079	23.93 ± 0.08
Throughput (amr/h)	21.08	$8.594\cdot10^{-3}$	0.038	21.08 ± 0.04
Utilisation S1	0.7162	$5.637\cdot10^{-6}$	0.0010	0.7162 ± 0.0010
Utilisation S2	0.9843	$5.786\cdot10^{-7}$	0.0003	0.9843 ± 0.0003
Utilisation S3	0.2811	$1.551\cdot10^{-5}$	0.0016	0.2811 ± 0.0016
Wait in buffer S1 (min)	0.868	$5.338\cdot10^{-5}$	0.003	0.868 ± 0.003
Wait in buffer S2 (min)	6.775	$2.081\cdot10^{-3}$	0.019	6.775 ± 0.019
Wait in buffer S3 (min)	2.019	$1.064\cdot10^{-2}$	0.043	2.019 ± 0.043
Total wait in hub (min)	7.689	$2.136\cdot10^{-3}$	0.019	7.689 ± 0.019

Table 2: Shows the outcome of the steady-state experiment with the simulation model generated by replicationMethodSimulation.py

Table 2 shows the sample average, sample variance and 95% confidence-interval half-width of each measure. It is computed from the simulation with 25 independent replications with T_0 = 120 min and T_E = 100 000 min [1]. The raw data is in experiment_raw_data.csv.

All half-widths in Table 2 are at most about 2% of their averages, so every measure in the table meets our error criterion of ε = 5% [1]. Only the redirection rate at S3 alone, which is not shown, does not, since S3 redirects an AMR only very rarely.
The bottleneck of the system is clearly the Stage 2 that have on average 98,43% Utilisation while both Stage 1 and Stage 3 are way less overloaded.

Measures	40 AMRs/h	45 AMRs/h	50 AMRs/h
Redirection rate (AMRs/h)	19.16 ± 0.09	23.93 ± 0.12	28.77 ± 0.19
Throughput (AMRs/h)	20.89 ± 0.07	21.10 ± 0.06	21.26 ± 0.07
Utilisation S1	0.6489 ± 0.0013	0.7164 ± 0.0016	0.7766 ± 0.0019
Utilisation S2	0.9741 ± 0.0005	0.9845 ± 0.0005	0.9906 ± 0.0003
Utilisation S3	0.2770 ± 0.0057	0.2779 ± 0.0041	0.2804 ± 0.0029
Total wait in hub (min)	7.20 ± 0.03	7.68 ± 0.02	8.09 ± 0.04

Table 3 the most important outcome from the validation.py where we used OFAT to validate by differing the arrival rate.

To verify our simulation model we choose to use the OFAT validation [4]. It works by changing parameters in the model and comparing the simulation data we get with how we expect the simulation data to change. For our experiment we choose to verify the system using the arrival rate. We compared the initial value of 45 AMR's per hour to 40 AMR's per hour and to 50 AMR's per hour since we can predict what would happen if we both increased and decreased arrival rate to stage 1.
When the arrival rate increased from 40 to 50 AMRs/h, the redirection rate increased by about 9,6 AMRs/h, while the throughput increased by only 0.4 AMRs/h, since s2 is almost at its limit. The confidence intervals of the three settings do not overlap for any measure except the S3 utilization, which is expected to stay constant. This proves that the model behaves as expected but further OFAT tests to other variables could be done to validate even more.

References

[1] Law, A.M. (2015) *Simulation modeling and analysis*. 5th edition. New York: McGraw-Hill Education.

[2] Buss, A. (2001) *Basic event graph modeling*, Simulation News Europe, Issue 31, April, pp. 1-6.

[3] Bank, J. et al. (2010) *Discrete-event system simulation*. 5th edition. Upper Saddle River, NJ: Pearson Prentice Hall.

[4] Caporuscio (2026) *" Building Valid, Credible, and Appropriately Detailed Simulation Models"* Simulation S2, 4DT901, LNU, slide 9

[5] Caporuscio (2026) "*Output Analysis"* Simulation S3, 4DT901, LNU, slide 5

AI Disclosure

In this assignment, we used AI to improve parts of the code, for example writing the results to CSV files, plotting the curves and formatting the printed output. We also used it in some cases to debug and refactor our code. The overall structure of the code is based on the example provided in the course, which we extended with our own features.